# Argumenty pozycyjne i nazwane

## Argumenty pozycyjne i nazwane

**Argumenty pozycyjne** (ang. *positional arguments*) — „zwykłe” przekazanie argumentu, to pod który parametr są podstawiane zależy od pozycji na liście argumentów w wywołaniu funkcji.

**Argumenty nazwane** (ang. *keyword arguments*) — podczas wywołania jawnie podane jest pod który parametr należy je podstawić.

In [2]:
```
print(3, 4, 5, sep=", ", end="; ")
print(6, 7, 8, end=".", sep=", ")
```

```
3, 4, 5; 6, 7, 8.
```

Pierwsze trzy argumenty w obu powyższych wywołaniach są argumentami pozycyjnymi, a ostatnie dwa — nazwanymi.

Poniższe przykłady prezentują użycie argumentów pozycyjnych i nazwanych.

In [3]:
```
def wyswietl_czas(godzina, minuta, sekunda):
    return f"{godzina:02}:{minuta:02}:{sekunda:02}"
```

Same argumenty pozycyjne:

In [4]:
```
wyswietl_czas(3, 4, 55)
```

Out[4]:
```
'03:04:55'
```

Godzina przekazana jest jako argument pozycyjny. Sekunda i minuta jako nazwane. Ich kolejność na liście argumentów nie ma znaczenia:

In [5]:
```
wyswietl_czas(3, sekunda=30, minuta=20)
```

Out[5]:
```
'03:20:30'
```

In [6]:
```
wyswietl_czas(3, minuta=20, sekunda=30)
```

Out[6]:
```
'03:20:30'
```

Same argumenty nazwane:

In [7]:
```
wyswietl_czas(sekunda=55, godzina=12, minuta=15)
```

Out[7]:
```
'12:15:55'
```

Argumenty pozycyjne muszą w wywołaniu występować przed nazwanymi. W poniższym zapisie po argumencie nazwanym `godzina=10` podano dwa argumenty pozycyjne, więc Python zgłosi wyjątek `SyntaxError` jeszcze przed wywołaniem funkcji:

In [8]:
```
wyswietl_czas(godzina=10, 10, 20)
```

```
  Cell In[8], line 1
    wyswietl_czas(godzina=10, 10, 20)
                                    ^
SyntaxError: positional argument follows keyword argument
```

## Argumenty nazwane i wartości domyślne

Mechanizm argumentów nazwanych dobrze współpracuje z parametrami z wartością domyślną. Zmieńmy teraz definicję funkcji `wyswietl_czas` (minuta i sekunda mają teraz domyślną wartość zero):

In [9]:
```
def wyswietl_czas(godzina, minuta=0, sekunda=0):
    return f"{godzina:02}:{minuta:02}:{sekunda:02}"
```

In [10]:
```
wyswietl_czas(12)
```

Out[10]: '12:00:00'

In [11]:
```
wyswietl_czas(godzina=12)
```

Out[11]: '12:00:00'

In [12]:
```
wyswietl_czas(6, sekunda=55)
```

Out[12]: '06:00:55'

In [13]:
```
wyswietl_czas(8, 15)
```

Out[13]: '08:15:00'

Wartości jednego parametru nie można podać dwukrotnie, nawet jeżeli za każdym razem jest taka sama. Tutaj wartość parametru `godzina` przekazujemy raz jako pierwszy argument pozycyjny, a drugi raz jako argument nazwany. Takie wywołanie spowoduje wyjątek `TypeError`:

In [14]:
```
wyswietl_czas(5, minuta=3, godzina=5)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[14], line 1
----> 1 wyswietl_czas(5, minuta=3, godzina=5)

TypeError: wyswietl_czas() got multiple values for argument 'godzina'
```

Wartość każdego parametru musi być podana przez argument pozycyjny, argument nazwany albo wartość domyślną. Parametr `godzina` nie ma wartości domyślnej, a w poniższym wywołaniu nie przekazano jego wartości. Z tego powodu otrzymamy wyjątek `TypeError`:

In [15]:
```
wyswietl_czas(minuta=3, sekunda=6)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[15], line 1
----> 1 wyswietl_czas(minuta=3, sekunda=6)

TypeError: wyswietl_czas() missing 1 required positional argument: 'godzina'
```

## Wymuszenie przekazywania argumentów jako pozycyjnych lub jako nazwanych

Przy porównywaniu sposobów wywołania użyjemy ponownie funkcji formatującej czas:

In [16]:
```
def wyswietl_czas(godzina, minuta=0, sekunda=0):
    return f"{godzina:02}:{minuta:02}:{sekunda:02}"
```

Przekazywanie argumentów jako argumentów nazwanych może poprawić czytelność wywołań funkcji. Zobaczmy to na przykładzie funkcji `datetime` z modułu `datetime`:

In [17]:
```
import datetime
```

W poniższym wywołaniu ciężko zorientować się, który parametr za co odpowiada:

In [18]:
```
dt = datetime.datetime(1, 2, 3, 4, 5, 6)
```

In [19]:
```
dt.minute
```

Out[19]: 5

In [20]:
```
dt.month
```

Out[20]: 2

Poniższe wywołanie jest czytelniejsze (ale niestety moduł `datetime` nie wymusza tego):

In [21]:
```
dt = datetime.datetime(year=1, month=2, day=3, hour=4, minute=5, second=6)
```

Używając wyłącznie argumentów pozycyjnych łatwo możemy omyłkowo przekazać parametry w złej kolejności:

In [22]:
```
h, m, s = 10, 5, 55
```

In [23]:
```
wyswietl_czas(s, m, h)
```

Out[23]:
```
'55:05:10'
```

## Wymuszenie przekazania argumentów jako nazwanych.

Funkcja może wymuszać wyłącznie argumenty nazwane dla wybranych parametrów. Np. funkcja `sorted` ma parametr `reverse` określający, czy kolejność sortowania ma być odwrócona:

In [26]:
```
help(sorted)
```

```
Help on built-in function sorted in module builtins:

sorted(iterable, /, *, key=None, reverse=False)
    Return a new list containing all items from the iterable in ascending or
der.

    A custom key function can be supplied to customize the sort order, and t
he
    reverse flag can be set to request the result in descending order.
```

In [24]:
```
sorted([3, 4, 5, 2, 0, 1, 2], reverse=True)
```

Out[24]:
```
[5, 4, 3, 2, 2, 1, 0]
```

Funkcja `sorted` przyjmuje tylko jeden argument pozycyjny. Próba przekazania wartości parametru `reverse` jako drugiego argumentu pozycyjnego zakończy się wyjątkiem `TypeError`:

In [25]:
```
sorted([3, 4, 5, 2, 0, 1, 2], True)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[25], line 1
----> 1 sorted([3, 4, 5, 2, 0, 1, 2], True)

TypeError: sorted expected 1 argument, got 2
```

Komunikat o błędzie oznacza, że oczekiwany był tylko jeden argument pozycyjny. Argument `reverse` musi być przekazany jako nazwany.

Wymuszenie takie realizuje się przez umieszczenie znaku `*` na liście parametrów. Wszystkie parametry po nim muszą być przekazane jako nazwane.

Poniższy przykład pokazuje kiedy może to być szczególnie przydatne. Zdefiniujmy najpierw funkcję bez użycia takiego wymuszenia.

In [27]:
```
def naglowek_wiadomosci(adresat, temat, pilne=False):
    if pilne:
        temat = f"!!PILNE!! {temat}"
    return f"Do: {adresat}\nTemat: {temat}\n"
```

In [28]:
```
print(naglowek_wiadomosci("Anna", "Zebranie"))
```

```
Do: Anna
Temat: Zebranie
```

In [29]:
```
print(naglowek_wiadomosci("Anna", "Zebranie", True))
```

```
Do: Anna
Temat: !!PILNE!! Zebranie
```

Co będzie jeżeli wiadomość będzie pilna, ale zapomnimy o temacie w wywołaniu funkcji:

In [30]:
```
print(naglowek_wiadomosci("Anna", True))
```

```
Do: Anna
Temat: True
```

Użyliśmy omyłkowo `True` jako wartości parametru `temat`. Nie została podana wartość parametru `pilne`, ale ma on wartość domyślną.

Poprawmy definicję. Teraz parametry `temat` i `pilne` muszą być przekazane jako nazwane:

In [31]:
```
def naglowek_wiadomosci(adresat, *, temat, pilne=False):
    if pilne:
        temat = f"!!PILNE!! {temat}"
    return f"Do: {adresat}\nTemat: {temat}\n"
```

Po zmianie definicji tylko adresat może być przekazany jako argument pozycyjny. Powtórzenie wcześniejszego problematycznego wywołania spowoduje więc wyjątek `TypeError`: drugi argument pozycyjny nie może już zostać przyjęty jako wartość parametru `temat`.

In [32]:
```
print(naglowek_wiadomosci("Anna", True))
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[32], line 1
----> 1 print(naglowek_wiadomosci("Anna", True))

TypeError: naglowek_wiadomosci() takes 1 positional argument but 2 were give
n
```

Poprawne wywołanie z jawnym tematem i oznaczeniem pilności:

In [33]:
```
print(naglowek_wiadomosci("Anna", temat="spotkanie", pilne=True))
```

```
Do: Anna
Temat: !!PILNE!! spotkanie
```

Kolejny przykład: funkcja `wyswietl_czas`, ale z wymuszeniem wszystkich argumentów nazwanych:

In [34]:
```
def wyswietl_czas(*, godzina, minuta, sekunda):
    return f"{godzina:02}:{minuta:02}:{sekunda:02}"
```

Wywołania są teraz czytelniejsze (wiadomo, który parametr za co odpowiada):

In [35]:
```
wyswietl_czas(godzina=10, minuta=5, sekunda=55)
```

Out[35]:
```
'10:05:55'
```

Trudniej o błąd z zamianą kolejności parametrów:

In [36]:
```
h, m, s = 10, 5, 55
```

Ta wersja funkcji `wyswietl_czas` wymaga, żeby wszystkie argumenty były nazwane. Wywołanie jej z trzema argumentami pozycyjnymi zostanie odrzucone z wyjątkiem `TypeError`, zamiast niepostrzeżenie zamienić godzinę z sekundą:

In [37]:
```
wyswietl_czas(s, m, h)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[37], line 1
----> 1 wyswietl_czas(s, m, h)

TypeError: wyswietl_czas() takes 0 positional arguments but 3 were given
```

Nie można teraz użyć argumentów pozycyjnych. Powyższe wywołanie musiałoby wyglądać tak (błąd jest tutaj od razu widoczny):

In [38]:
```
wyswietl_czas(godzina=s, minuta=m, sekunda=h)
```

Out[38]: '55:05:10'

### Wymuszenie przekazania argumentów jako pozycyjnych

Czasem nazwa danego parametru nie jest istotna i nie chcemy, żeby była widoczna w wywołaniach funkcji. Jeżeli nie jest istotna, to mogła np. nie być dobrze przemyślana i chcielibyśmy ją zmienić w definicji funkcji. Jeżeli nazwa ta mogła być użyta w wywołaniach z argumentem nazwanym, to po jej zmianie musimy ją również zmienić we wszystkich takich wywołaniach.

Jeżeli chcemy, żeby dało się użyć wyłącznie argumentu pozycyjnego dla danego parametru, to musimy go umieścić przed znakiem `/` na liście parametrów.

Przykład: obliczenie długości przedziału. Zadane są końce przedziału w dowolnej kolejności. Nazwy końców przedziałów nie są istotne:

In [39]:
```
def dlugosc_przedzialu(a, b, /):
    return abs(a - b)
```

In [40]:
```
dlugosc_przedzialu(4.5, 3)
```

Out[40]: 1.5

Parametry `a` i `b` poprzedzają separator `/`, więc ich wartości trzeba podać pozycyjnie. Użycie ich nazw w poniższym wywołaniu spowoduje wyjątek `TypeError`:

In [41]:
```
dlugosc_przedzialu(a=4.5, b=3)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[41], line 1
----> 1 dlugosc_przedzialu(a=4.5, b=3)

TypeError: dlugosc_przedzialu() got some positional-only arguments passed as
keyword arguments: 'a, b'
```

Kolejny przykład: funkcja obliczająca średnią ważoną.

Parametry:

- `dane` — sekwencja liczb (wyłącznie argument pozycyjny),
- `wagi` — wagi do średniej ważonej (argument pozycyjny lub nazwany),
- `dokladnosc` — jeżeli jest podany, to wynik jest zaokrąglany (wyłącznie argument nazwany).

Przykład średniej zakłada niepuste dane, taką samą liczbę wag jak danych i niezerową sumę wag. Skupiamy się na rodzajach parametrów, nie na walidacji danych wejściowych.

In [42]:
```
def srednia(dane, /, wagi=None, *, dokladnosc=None):
    if wagi is None:
        wagi = [1] * len(dane)
    wynik = sum(d * w for d, w in zip(dane, wagi)) / sum(wagi)
    if dokladnosc is not None:
        wynik = round(wynik, dokladnosc)
    return wynik
```

Przykłady poprawnego użycia:

In [43]:
```
srednia([1, 2])
```

Out[43]: 1.5

In [44]:
```
srednia([1, 2], [1, 7])
```

Out[44]: 1.875

In [45]:
```
srednia([1, 2, 2, 2, 2, 2, 2], dokladnosc=4)
```

Out[45]: 1.8571

In [46]:
```
srednia([1, 2], [1, 7], dokladnosc=1)
```

Out[46]: 1.9

In [47]:
```
srednia([1, 2], wagi=[1, 7])
```

Out[47]: 1.875

In [48]:
```
srednia([1, 2], wagi=[1, 7], dokladnosc=1)
```

Out[48]: 1.9

In [49]:
```
srednia([1, 2], dokladnosc=1, wagi=[1, 7])
```

Out[49]: 1.9

Parametr `dane` musi otrzymać wartość jako argument pozycyjny. Próba przekazania jej pod nazwą `dane` zakończy się wyjątkiem `TypeError`:

In [50]:
```
srednia(dane=[1, 2])
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[50], line 1
----> 1 srednia(dane=[1, 2])

TypeError: srednia() got some positional-only arguments passed as keyword ar
guments: 'dane'
```

Wartość parametru `dokladnosc` można przekazać wyłącznie jako argument nazwany. W poniższym wywołaniu podano ją jako trzeci argument pozycyjny, którego funkcja nie przyjmuje; otrzymamy wyjątek `TypeError`:

In [51]:
```
srednia([1, 2], [1, 7], 1)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[51], line 1
----> 1 srednia([1, 2], [1, 7], 1)

TypeError: srednia() takes from 1 to 2 positional arguments but 3 were given
```